

Codebase Tour

A file-by-file guide to the project: what every source file is for, the four request paths traced hop by hop with line numbers, and an index from "what someone might ask" to "the file to open".

AUTHOR	Tom
REPOSITORY	github.com/TomKoren1/Sightline
COMPANION TO	sightline-handover.pdf (the decisions and the problems)
PURPOSE	Answering questions about the code with the code open

13.2k

LINES, EXCL. TESTS

80

SOURCE FILES

253

UNIT TESTS

4

TRACED PATHS

Contents

1 How to use this document

The eight files that matter most

Reading orders: ten minutes, an hour

2 The shape of the repository

The tree, with sizes

Why the boundaries fall where they do

3 Trace: a scan, end to end

4 Trace: a question, end to end

5 Trace: connecting a real AWS account

6 Trace: a finding becomes a fix

7 The data model

Twenty node kinds, thirteen edge types

Postgres tables, and what is derived

8 File reference — apps/api

aws/ · scan/ · db/ · agent/ · remediation/ · evals/ · cli/ · health/ graph/ scans/ chat/ connection/

9 File reference — apps/web

10 File reference — packages and infra

11 The test suite, file by file

12 If they ask X, open Y

13 Worth knowing without looking it up

1 How to use this document

The handover PDF explains *why* this project is the way it is. This one explains *where things are*. It is written to be open on a second screen while someone asks questions about the code, so every claim in it carries a file and, where it helps, a line number.

Line numbers were correct when this was generated and drift as the code changes. Treat them as "near the top" / "about two thirds down" rather than as exact addresses, and use the symbol names, which do not move.

The eight files that matter most

Eighty source files and twenty-one test files, and this is where the substance is. If someone reads nothing else, read these.

File	Why it matters
<code>scan/runner.ts</code>	Scan orchestration, and the one design commitment that shapes the whole product: a scan does not fail, it reports per-unit outcomes.
<code>scan/pipeline.ts</code>	Where raw AWS JSON becomes verdicts. Every security fact in the product is computed here, over the whole estate at once, never by the model.
<code>scan/analysers/publicAccess.ts</code>	The best single example of why the model is not allowed to answer: four inputs interact, and three of them can cancel a fourth.
<code>db/queries.ts</code>	The curated query library — thirteen hand-written Cypher queries that are the only questions the agent can ask.
<code>agent/tools.ts</code>	The tool boundary: sixteen parameterised tools over those queries. The model picks and fills in arguments; it never writes a query.
<code>agent/citations.ts</code>	Every ARN in an answer is checked against what the tools actually returned. This is the answer to "why should we trust it".
<code>infra/readonly-role.yaml</code>	The replacement read-only role: <code>SecurityAudit</code> + <code>ViewOnlyAccess</code> , an explicit deny on data-plane reads, trust scoped to one principal with an external id.
<code>packages/mock-aws/src/topology.ts</code>	The hand-written answer key. Ground-truth tests grade the scanner against this, so an analyser bug cannot mark its own homework.

Reading orders

Ten minutes

`packages/shared/src/model.ts` (what a resource is) → `scan/runner.ts` (how one is found) → `scan/pipeline.ts` (how it gets a verdict) → `agent/tools.ts` (what the model may ask).

An hour

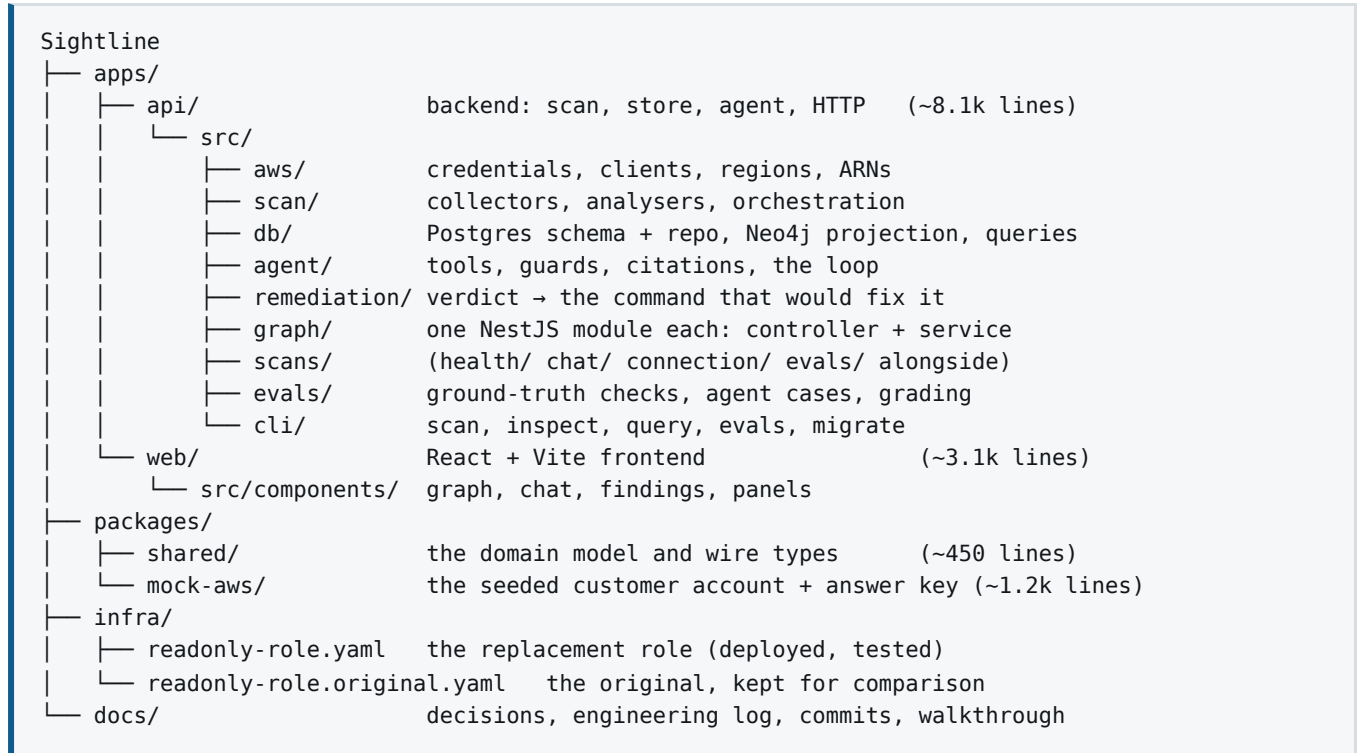
The four traces in sections 3–6, in order, with the files open. They cover every layer and cross every boundary in the system, which no single file does.

THE ONE-SENTENCE ARCHITECTURE

The scanner reads AWS and writes Postgres; Postgres is the system of record and the history; Neo4j is a projection rebuilt from it; the agent reads only Neo4j, only through curated tools; the UI reads the API and never AWS.

2 The shape of the repository

An npm workspace monorepo: two applications, two shared packages, one infrastructure directory. TypeScript throughout, so the resource model is one type definition shared by the scanner, the API, the agent and the React components (ADR-001).



Why the boundaries fall where they do

packages/shared exists so the model cannot drift. A resource kind is one `as const` array; adding a kind without handling it in the graph projection or the UI colour map is a type error rather than a runtime surprise.

aws/ is separate from **scan/** because one is about *how we talk to AWS* (assume-role, retries, endpoints, region discovery) and the other is about *what we ask it*. The only difference between scanning the mock and scanning a real account lives in `aws/clients.ts` — an endpoint override — which is why the same scanner code is exercised by both.

Collectors never interpret. A collector returns resources and the relationships it can see locally. Anything that needs two resources to be true at once — "this instance is exposed", "this role is admin", "this volume is idle" — is computed later, in `scan/pipeline.ts`, over the whole estate. That split is why a security verdict can be unit-tested without AWS, a database, or a network.

db/queries.ts is the agent's only vocabulary. The agent directory holds no Cypher except the guarded escape hatch. Someone reviewing what the agent can possibly do reads one file.

packages/mock-aws is a workspace, not a fixture directory, because the seeder builds the account through the real AWS SDK against moto. The resources the scanner later discovers

were produced by the same API surface AWS would produce, not hand-written JSON that happens to match what the code expects.

WHERE THE MOCK BOUNDARY ACTUALLY IS

Exactly one place: `aws/clients.ts` sets an `endpoint` override and static credentials when `AWS_MODE=mock`. Every other file — collectors, analysers, pipeline, repository, projection — is identical in both modes and untestable against one but not the other. The scanner cannot tell the difference, which is the point.

3 Trace: a scan, end to end

Triggered by `npm run scan` or by `POST /api/scans` from the UI. The two entry points do the same work in the same order; the route additionally streams progress.

- 1 `cli/scan.ts:26 · scans/scans.controller.ts:75`
Entry. `migrate()` first, so the schema exists, then `runScan()`. *The route creates the scan row and opens a server-sent event stream before doing any work, so the UI can show progress from the first unit.*
- 2 `scan/runner.ts:102 → aws/credentials.ts:107`
`getSession()` assumes the customer role with an external id and caches the credentials until they are near expiry. Concurrent callers during a refresh share one in-flight `AssumeRole` rather than stampeding STS — a scan fans out across every region at once and they all miss the cache in the same millisecond. *This is where the account id comes from — it is read from the assumed session, never from configuration, which is what stops a mock scan being labelled with a real account's number.*
- 3 `aws/regions.ts`
Region discovery via `DescribeRegions`, filtered to what is actually enabled, intersected with `AWS_SCAN_REGIONS` when set. *Guessing regions wastes an API call per service per region and still misses the one the customer actually uses.*
- 4 `scan/runner.ts:128`
The unit list is built: one `(service, region)` pair per regional service, one global unit for IAM and S3. Six services over three regions is fourteen units.
- 5 `scan/runner.ts:197 → scan/limiter.ts:12`
`runWithConcurrency(tasks, SCAN_CONCURRENCY)`. *Unbounded fan-out on a real account is a few hundred simultaneous AWS calls and guaranteed throttling. Thirty lines: a fixed pool of workers pulling from a shared index, and no rejection handling, because a failed unit is an outcome rather than a thrown error.*
- 6 `scan/runner.ts:175 → scan/collectors/*.ts`
Each unit calls one collector. Each returns resources plus the relationships visible locally. A collector that throws fails *its own unit only*: `explainFailure()` at line 71 turns the SDK error into something a user can act on (`AccessDenied` becomes "Not permitted to read rds in eu-west-1. The role is missing a Describe/List permission for this service."), and the scan continues.
- 7 `scan/runner.ts:200 → scan/pipeline.ts:64`
`annotate()` runs once over the merged estate: synthetic `Account`, `Region` and `Internet` nodes, then the four analysers in order — admin (line 117), public access (123), idle (142), reachability (145).
- 8 `db/repository.ts:45`
`saveScanResult()` writes units, resource snapshots and relationship snapshots in one transaction. *All or nothing: a half-written scan would be worse than no scan, because it would read as complete.*
- 9 `db/neo4j.ts:139`
`projectGraph()` rebuilds the graph wholesale from that scan's snapshots, inside one transaction. *Postgres is written first deliberately: if the projection fails the scan is still durable and can be re-projected without going back to AWS.*

THE QUESTION THIS TRACE ANSWERS

"What happens if one region is throttled or one permission is missing?" Nothing fails. The unit fails, the banner names it, the other thirteen units still produce a graph, and the scan is recorded as `partial` rather than `succeeded` — computed by `rollUpStatus()` in `packages/shared/src/scan.ts`, which has its own tests.

4 Trace: a question, end to end

"Which S3 buckets are public?" typed into the chat panel. This is the path most worth being able to narrate, because every AI-engineering decision in the project is visible along it.

- 1 `web/components/Chat.tsx:88 → web/src/api.ts:203`
`POST /api/chat` over `fetch`, reading a server-sent event stream from the response body. *Not `EventSource`, which cannot send a request body — noted in `web/src/api.ts` where the reader lives.*
- 2 `chat/chat.controller.ts:29`
Opens the stream, then calls `ask()` with an `emit` callback. Every event the agent produces is forwarded to the browser as it happens; the final message is also persisted to the `messages` table.
- 3 `agent/agent.ts:80`
`ask()`. First it loads the latest scan and an account summary and puts them in the system prompt, so the model knows when the data is from and how big the account is before it calls anything.
- 4 `agent/agent.ts:116`
The loop, capped at `MAX_ITERATIONS`: stream a model turn, collect tool-use blocks, run them, hand the results back, repeat until the model answers in prose. Text blocks emitted *during* tool-calling turns become `agent.thinking` events rather than being dropped.
- 5 `agent/tools.ts:273`
`runTool()` dispatches by name to one curated query — `find_public_resources` → `q.findPublicResources()`. Sixteen cases, one `graph_query` escape hatch. *The model chose the tool and the arguments; it did not write the query.*
- 6 `db/queries.ts:88`
Hand-written Cypher, run in a Neo4j *read* transaction. It matches on `isPublic = true` — a property computed during the scan — and returns the `publicReason` alongside it, so the answer can quote evidence rather than assert a verdict.
- 7 `agent/citations.ts:122`
Every ARN in every tool result is recorded in a `CitationTracker` as the loop runs.
- 8 `agent/agent.ts:231 → agent/readOnlyGuard.ts:134`
`enforceReadOnlyNotice(question, answer)`. If the question asked for a change (`requestsMutation()`, line 105) and the answer does not already decline (`assertsReadOnly()`, line 121), the refusal is prepended. *It runs on the finished answer rather than short-circuiting the question, because the model's answer is usually genuinely useful — it names the resource, its state and the command an engineer would run. Only the missing sentence is added. The prompt asks for this too and the model does not reliably comply, which is exactly why it is code (ADR-009).*
- 9 `agent/citations.ts:59`
`validateCitations()` extracts every ARN-shaped string from the finished answer and checks each against what the tools actually returned. Unknown ARNs are reported, not silently kept. *This is the mechanism behind "how do you know it isn't making things up", and it caught a real hallucination — engineering log #32, where a tool returned paths but not the resource they started from, and the model filled the gap with a plausible truncated ARN.*
- 10 `agent/agent.ts:249 → Chat.tsx:113`
`agent.finished` carries the message, its tool-call trail and its validated citations. The UI highlights every cited ARN in the graph and keeps the trail expandable under the answer.

FOUR GUARANTEES ALONG ONE PATH

The role cannot write; the scanner only calls `Describe / List / Get`; no tool can express a mutation; raw Cypher is lexically validated (`agent/cypherGuard.ts`) *and* run in a read transaction. Independent layers, so no single mistake removes the guarantee.

5 Trace: connecting a real AWS account

The onboarding flow. Its design rule is that the product **guides** onboarding and never **performs** it: nothing here deploys a stack or writes a customer's configuration (ADR-010).

- 1 [web/components/ConnectionGuide.tsx](#)
Five steps with copyable commands, each block followed by a `<Fields>` legend marking which values are pre-filled and which the reader supplies. Opened from the header, and also linked from the empty state.
- 2 [connection/connection.service.ts:243](#)
`GET /api/connection/external-id` generates a per-customer external id. *The customer pastes it into their own stack; it is a shared secret that prevents the confused-deputy problem, not a password.*
- 3 [connection/connection.service.ts:120](#) → [aws/principal.ts:49](#)
`GET /api/connection` reports who this backend runs as, converted for a trust policy. `assumablePrincipalArn()` exists because `sts:GetCallerIdentity` answers a *different question* from the one a trust policy asks: it returns the **session** ARN (`arn:aws:sts::...:assumed-role/Role/session`), and a trust policy needs the **identity** (`arn:aws:iam::...:role/Role`). The guide showed the wrong one until a real deployment failed on it.
- 4 [aws/principal.ts:156](#)
`validateAssumeRoleTarget()` checks the configured target is a role, because `sts:AssumeRole` cannot assume a *user* — the exact mix-up a customer makes when they paste their own identity into the scanner's configuration. It names the mistake instead of failing later with `AccessDenied`.
- 5 [connection/connection.service.ts:254](#)
`POST /api/connection/test` attempts the assume-role and *diagnoses* the failure: trust policy names a different principal, external id mismatch, role does not exist, wrong account. Each maps to a specific next action rather than echoing the SDK error.
- 6 [connection/connection.service.ts:206](#)
`POST /api/connection/mode` switches between the demo account and the real one at runtime. *It does not rewrite `.env`, and it does not rescan — the banner says the graph still holds the previous account's data until you run one (ADR-013).*

The stack the customer deploys is `infra/readonly-role.yaml`, and it has its own test file — `infra/template.test.ts`, twenty-two assertions — because the original template turned out never to have been deployed. That file checks the permission set, the explicit deny, the trust scoping, the external-id condition, and the two CloudFormation length limits that a real deployment rejected.

6 Trace: a finding becomes a fix

Clicking a flagged node opens the verdict and, beside it, the exact commands that would fix it. Nothing in the product runs them (ADR-014). This trace is short but crosses three boundaries, and the last one is the interesting one.

- 1 `scan/pipeline.ts:123 → analysers/publicAccess.ts`
At scan time, the verdict and its reason are written onto the resource as `derived.isPublic` / `derived.publicReason`. *The fix and the verdict therefore come from the same evidence, computed once.*
- 2 `db/neo4j.ts:139`
The projection promotes derived facts to top-level node properties, and JSON-encodes nested ones. *Neo4j properties must be primitives or arrays of primitives, so a public access block or a list of ingress rules is stored as a string.*
- 3 `graph/graph.controller.ts:37`
`GET /api/resources/:arn/remediation`. *This route and its sibling are why `maxParamLength` is 2048 in `app.ts`: an ARN is the path parameter, and a service-linked role's ARN is 120 characters — over Fastify's default cap of 100, which returned 414 for ten of nineteen IAM roles in a real account.*
- 4 `remediation/fromGraph.ts:87`
`remediationInputFromGraph()` parses those JSON-string properties back into objects, and filters incoming edges to the four that mean "in use". *Parsing is best-effort by design: a property that will not parse is dropped rather than thrown on, because a 500 on a detail page is worse than one missing suggestion.*
- 5 `remediation/remediation.ts:529`
`remediationsFor(input)` — one generator per finding kind, ordered public → exposed → admin → unprotected → idle. Each returns commands, a `caution` (a required field, not optional), a risk rating, and a read-only command to verify the fix worked.
- 6 `web/components/Remediation.tsx`
Rendered with a copy button and **no apply button**. The caution is rendered *above* the commands, because a warning under the thing it warns about is decoration.

THE DETAIL WORTH SHOWING SOMEONE

`LegacyDeployRole` in the fixture is an administrator by way of an *inline* policy named `legacy-deploy-inline` — no `AdministratorAccess` attached. The generator selects the managed-policy detach only when a policy ARN actually matches (line 201), so for this role it emits none, and instead produces "replace the inline policy": save the current document to `backup.json` with `get-role-policy`, write a scoped replacement, then `put-role-policy`. The obvious `detach-role-policy --policy-arn .../AdministratorAccess` would have exited zero and changed nothing.

The `caution` is generated from the graph too: the edges into the role name what uses it, so the warning reads "*legacy-image-resizer* currently uses this role, and will lose every permission it grants the moment this is applied". For a *user*, the text deliberately says the opposite — nothing visibly using it is not evidence it is unused, because access keys live in CI pipelines and on laptops where an inventory cannot see them.

7 The data model

One type definition, in `packages/shared/src/model.ts`, shared by the scanner, the projection, the agent's queries and the React components. Twenty node kinds and thirteen edge types (ADR-003).

Node kinds

`Account` · `Region` · `Vpc` · `Subnet` · `SecurityGroup` · `InternetGateway` · `NatGateway` · `RouteTable` · `Ec2Instance` · `EbsVolume` · `ElasticIp` · `S3Bucket` · `IamRole` · `IamUser` · `IamPolicy` · `InstanceProfile` · `RdsInstance` · `DbSubnetGroup` · `LambdaFunction` · `Internet`

`Account`, `Region` and `Internet` are synthetic — they are not AWS resources. `Internet` exists so that "what can reach the production database?" is a path query from a node rather than a special case in code.

Edge types

Edge	Meaning
<code>IN_REGION</code> <code>IN_VPC</code> <code>IN_SUBNET</code>	Containment. Cheap, and what makes region and tier questions trivial.
<code>HAS_SECURITY_GROUP</code> <code>ROUTES_TO</code> <code>USES_SUBNET_GROUP</code>	Network structure as AWS reports it, before any interpretation.
<code>ATTACHED_TO</code>	Volume to instance, elastic IP to instance. Absence is the idle signal.
<code>HAS_INSTANCE_PROFILE</code> <code>PROVIDES_ROLE</code> <code>EXECUTES_AS</code> <code>HAS_POLICY</code> <code>CAN_ASSUME</code>	The identity chain. <code>EXECUTES_AS</code> is how a trivial Lambda turns out to be running as an administrator.
<code>CAN_REACH</code>	Derived, not observed. Computed by <code>analysers/reachability.ts</code> from security group rules, subnet routing and public-accessibility flags, carrying the ports and the evidence that justified it.

Derived facts

Every verdict is a flag paired with a human-readable reason, because the agent is instructed to quote the reason — a DevOps engineer needs to be able to disagree with a finding, which means seeing the evidence.

Field	Question it answers
<code>isPublic</code> / <code>publicReason</code>	Reachable by an anonymous principal, or from the internet.
<code>isUnprotected</code> / <code>unprotectedReason</code>	Nothing <i>prevents</i> this becoming public. Deliberately separate from <code>isPublic</code> : a bucket with Block Public Access off is not public, it has lost a guardrail (ADR-012).
<code>isAdmin</code> / <code>adminReason</code>	Effective <code>*:*</code> , by any combination of managed and inline policy.
<code>isIdle</code> / <code>idleReason</code> / <code>estimatedMonthlyCostUsd</code>	Provisioned, billable, structurally unused.

Postgres tables

In `apps/api/src/db/schema.ts`. Everything is append-only per scan — a scan never updates a previous scan's rows, which is what makes change detection answerable and lets a bad scan be discarded without corrupting history.

Table	Holds
<code>scan_runs</code>	One row per scan: account, status, regions, counts, API calls.
<code>scan_units</code>	One row per <code>(service, region)</code> — the unit of partial failure, and the table the banner reads. <code>region</code> is <code>'global'</code> rather than NULL so it can be part of the primary key.
<code>resource_snapshots</code>	Immutable snapshot per resource per scan, with a <code>fingerprint</code> hash of the queryable state — so diffing two scans is an index-backed join, not a JSONB comparison across thousands of rows.
<code>relationship_snapshots</code>	The edges as of that scan.
<code>conversations</code> / <code>messages</code>	Chat history, plus the audit trail: which tools ran, which ARNs were cited, and whether any citation was unsupported. Kept out of the graph so chat volume never competes with the agent's own queries.
<code>eval_runs</code>	Eval results over time, so a regression in answer quality is visible rather than discovered in a demo.

THE SENTENCE THAT ANSWERS "WHY TWO DATABASES?"

Postgres answers "what did this account look like at 09:00, and what changed since?" — a history question, and relational storage with an immutable snapshot per scan is the right shape. Neo4j answers "what can reach the production database?" — a variable-length path question, which is three lines of Cypher and a recursive CTE nobody enjoys reviewing. The graph is disposable; it is rebuilt from Postgres, never written to directly.

8 File reference — apps/api

Every non-test file, with the thing worth saying about it. Sizes are lines including the header comments, which in this codebase carry the reasoning.

`src/aws/` — how we talk to AWS

File	Ln	What it is for
<code>credentials.ts</code>	143	Cross-account assume-role with an external id. Sightline never holds customer keys. Caches the session, shares one in-flight refresh between concurrent callers, and takes the account id from the assumed session rather than from configuration.
<code>clients.ts</code>	118	Every AWS client, built one way. <code>retryMode</code> : "adaptive" (client-side rate limiting on top of exponential backoff with jitter) and an API call counter. The only place that knows whether we are talking to moto or to AWS.
<code>regions.ts</code>	43	Region discovery from <code>DescribeRegions</code> , so disabled regions cost nothing and enabled ones are not missed.
<code>arns.ts</code>	46	ARN construction and tag normalisation. Most EC2 APIs return bare ids (vpc-0abc), and the graph needs one stable identity per resource, so we build the ARN AWS would have given.
<code>principal.ts</code>	176	Session ARN → trust-policy principal, and validation that an assume-role target is a role rather than a user. Written after a real deployment failed on the distinction.
<code>resourceExplorer.ts</code>	144	A fast path, used as one. Needs an index that only exists if the customer created it — and a read-only role cannot create one — so SDK enumeration stays the tested default.

File	Ln	What it is for
runner.ts	205	Orchestration. Builds the unit list, runs it under the limiter, turns SDK errors into actionable text, emits progress events. A scan does not fail.
pipeline.ts	161	annotate(): synthetic nodes, then the four analysers over the whole estate. Cross-resource facts live here because they cannot be known one resource at a time.
limiter.ts	30	Bounded concurrency. A fixed worker pool over a shared index, no rejection handling.
collectors/vpc.ts	303	The network substrate every reachability answer is built from: VPCs, subnets, route tables, gateways, security group rules.
collectors/iam.ts	323	Does the most work per resource, because "which roles have admin?" needs policy documents, not names. Roles and users, managed and inline, with a shared managed-policy cache.
collectors/ec2.ts	194	Instances, volumes, elastic IPs — collected <i>especially</i> when attached to nothing, since that is the clearest waste signal.
collectors/rds.ts	122	Instances and subnet groups. Records PubliclyAccessible but deliberately does not treat it as the answer to "is this exposed?".
collectors/s3.ts	120	Buckets. Global namespace, so one unit, resolving each bucket's home region individually.
collectors/lambda.ts	74	Functions, and the execution-role edge — the one that turns an image resizer into an administrator.
collectors/types.ts	29	The contract every collector implements, plus output merging.
analysers/publicAccess.ts	177	Four interacting inputs: bucket policy, ACL, Block Public Access, and account-level settings. Produces both the exposure verdict and the separate posture verdict.
analysers/reachability.ts	187	Security group rules and routing → CAN_REACH edges, with ports and evidence, so paths become a query rather than a code path.
analysers/policy.ts	100	IAM policy evaluation. Deliberately small and conservative — not a full evaluator, and the header says which constructs it does not model.
analysers/idle.ts	134	Structural waste only: attached to nothing, associated with nothing, stopped. No usage metrics, so no claim the data cannot support.

src/db/ — storage

File	Ln	What it is for
schema.ts	160	Seven tables as Drizzle definitions, append-only per scan, with the reasoning in comments. drizzle/ holds the generated migrations; the baseline one also adopts a database built by the SQL schema it replaced.
postgres.ts	70	Pool and migration. One idempotent schema file, applied at startup.
repository.ts	344	Reads and writes scans. Batched inside a transaction — all or nothing. Also diffScans(), which is a fingerprint join rather than a JSONB comparison.
neo4j.ts	211	The projection: rebuilt wholesale per scan in one transaction, derived facts promoted to top-level properties, nested values JSON-encoded because Neo4j takes only primitives.
queries.ts	386	Thirteen hand-written queries — the agent's entire vocabulary. Anyone auditing what the agent can do reads this one file.

src/agent/ — the AI layer

File	Ln	What it is for
agent.ts	251	The tool-calling loop, on the Vercel AI SDK. Streams tokens, tool calls and tool results. Each tool's execute records into the citation ledger before the rows reach the model.
tools.ts	375	Sixteen tool definitions plus the dispatch. The model picks and parameterises; it never writes a query.
citations.ts	151	The trust mechanism: every ARN in an answer checked against what the tools returned. Unsupported citations are surfaced as warnings, not swallowed.
cypherGuard.ts	127	An allowlist of opening clauses plus a denylist of write keywords, applied after string literals and comments are stripped — so a node named "DELETE ME" cannot trip it and, more importantly, a write cannot hide inside a string. Lexical, not a parser, and deliberately strict: a wrong rejection costs one retry, a wrong acceptance breaks the read-only rule.
readOnlyGuard.ts	144	Guarantees a change request is answered with an explicit refusal, keeping the useful part of the answer.
prompt.ts	95	The system prompt, written for one reader: an engineer about to act. Instructs the model to quote computed reasons rather than reason about policy itself.

src/remediation/ , src/evals/ , src/cli/

File	Ln	What it is for
remediation/remediation.ts	541	One generator per finding kind. caution is a required field. Nothing executes.
remediation/fromGraph.ts	120	The seam that parses JSON-string properties back, so the generators stay pure functions of a plain object and testable without a database.
app.ts	124	The NestJS app on a Fastify adapter, with no listener, so tests can inject(). Carries maxParamLength: 2048; the health endpoint that explains a missing API key is now health/health.service.ts.
server.ts	27	Build, migrate, listen, handle signals. Nothing else.
config.ts	368	Zod-validated environment, read once. Treats a blank value as unset, resolves the .env path with fileURLToPath, and reports configuration problems as text the UI can show.
scans/scans.controller.ts	126	Scan history, the diff, and the streaming POST /api/scans.
graph/graph.controller.ts	63	Graph, summary, resource detail, remediation, search, findings — all over curated queries.
chat/chat.controller.ts	114	The agent's event stream, and conversation persistence.
evals/evals.controller.ts	141	Runs the ground-truth checks on demand and reports the last agent eval run — the Trust panel's backend.
connection/connection.service.ts	307	Onboarding: identity, external id, connection test with diagnosis, runtime mode switch.
evals/checks.ts	349	Ground-truth checks as <i>data</i> , so the same definitions run from the test suite and from the product's Trust panel.
evals/cases.ts	269	Twenty-one agent eval cases, including the six headline questions verbatim.
evals/grade.ts	128	Scores on cited ARNs (precision, recall, F1) rather than text similarity, because citations are already validated.
cli/*.ts	~400	scan, inspect (scan and print findings with no databases), query (run every curated query), evals, migrate.

9 File reference — apps/web

React, Vite, TanStack Query, React Flow (xyflow), Tailwind. About 3,100 lines. The frontend holds no security logic at all — it renders verdicts computed during the scan, which is why the same finding reads identically in the UI, in the agent's answer and in the CLI.

File	Ln	What it is for
App.tsx	287	The shell and all shared state: banner on top, findings left, graph centre, chat right. The graph is the shared surface — both the sidebar and the agent highlight nodes on it. Queries for graph, summary and findings are enabled: hasScan, so an empty database makes no pointless requests and the empty state stays clean.
api.ts	258	Thin client. Holds the SSE reader used by both scan and chat — EventSource cannot send a request body, so both stream over fetch.
components/GraphView.tsx	198	React Flow. Nodes carry their verdict, so an exposed instance is visible without clicking, and cited nodes are highlighted.
components/Chat.tsx	370	Names the running tool in plain language rather than showing a spinner, streams tokens, and keeps an expandable tool-call trail under every answer.
components/ScanBanner.tsx	172	Six of the seven UX states in one component — progress, freshness, staleness, refresh, partial failure, error — because they are all the same question: can I trust this?
components/Findings.tsx	243	Standing answers to the questions the tool exists to answer, available without typing. Clicking a row highlights it in the graph.
components/ChangesPanel.tsx	240	The scan diff, separating material change from bookkeeping. Built server-side long before it was visible here, which was the wrong way round.
components/ResourceDetail.tsx	218	Properties, verdicts with their reasons, neighbours, and the remediation panel.
components/Remediation.tsx	134	The fix, with a copy button and no apply button. The caution renders above the commands.
components/TrustPanel.tsx	243	"How much should I trust this?" answered inside the product: the ground-truth checks run on demand with no API key, plus the last agent eval run.
components/ConnectionGuide.tsx	337	Four-step onboarding with copyable commands and a diagnosing connection test.
components/ModeToggle.tsx	110	Demo ↔ real account at runtime, with a visible reason when it is unavailable rather than a dead control.
components/Copyable.tsx	49	Command block plus copy button, shared by onboarding and remediation. The button sits above the block, not over it.
components/Modal.tsx	56	Overlay; closes on backdrop click and Escape.
layout.ts	49	Dagre, left to right: internet on the left, database at the end of the chain. A force layout looks livelier and makes that relationship harder to see.
kinds.ts	49	Colour and icon per resource kind, where colour carries meaning — network muted, identity set apart, because IAM is not part of the network picture.

10 File reference — packages and infra

packages/shared

File	Ln	What it is for
model.ts	164	Resource kinds, edge types, derived facts, the Resource and Relationship shapes. The single source of truth for what this product thinks an AWS account is.
scan.ts	155	Scan lifecycle types, the service and global-service lists, and rollUpStatus() — which decides succeeded / partial / failed and has its own tests, because that verdict is what the whole partial-failure story rests on.
remediation.ts	75	The Remediation shape, with caution required rather than optional, plus shellQuote().
agent.ts	56	The agent wire types — every event the stream can carry, shared by API and UI.

packages/mock-aws

File	Ln	What it is for
topology.ts	155	The answer key , written by hand rather than generated from the code under test — so an analyser bug cannot grade itself correct. Ground-truth tests compare the scanner's output against this.
seed.ts	780	Builds the account through the real AWS SDK against moto: three regions, two buckets with identical policies and different outcomes, a private database reachable two ways, a public database reachable none, three admin roles, two admin users, and about \$103/month of waste.
drift.ts	156	Simulates a day passing — a bucket goes public, a port opens, an instance stops — so change detection has something real to detect. Re-seeding would not do: it would change everything at once.
clients.ts	56	Static credentials, deliberately not the scanner's assume-role path: conceptually the seeder <i>is</i> the customer, not Sightline.

File	Ln	What it is for
readonly-role.yaml	326	The replacement role: SecurityAudit + ViewOnlyAccess, an explicit Deny on data-plane reads (s3:GetObject, secretsmanager:GetSecretValue, sqs:ReceiveMessage and friends), trust scoped to the scanner principal with an external-id condition, and sts:SourceIdentity so the customer can attribute every scan in their own CloudTrail. Deployed and tested, not written and hoped over.
readonly-role.original.yaml	83	The original template, kept for side-by-side comparison.
template.test.ts	220	Twenty-two assertions over the template: the permission set, the deny list, trust scoping, the external-id condition, and the two CloudFormation length limits a real deployment rejected.

11 The test suite, file by file

253 tests across 21 files. Two tiers plus a set of guards. What is unusual here is not the count — it is that several of these tests exist to catch a *class* of mistake that was actually made, rather than to cover a line.

File	#	What it guards
remediation/remediation.test.ts	35	That each generator reads the real evidence — including that an inline admin grant does not produce a managed-policy detach — and that shell quoting survives a bucket name with a quote in it.
infra/template.test.ts	22	The CloudFormation template, including two length limits a real deployment rejected. Written because the original template had never been deployed.
analysers/publicAccess.test.ts	20	The four-input interaction, in both directions: a permissive policy that <i>is</i> public, and a byte-identical one that is not.
agent/citations.test.ts	19	That an ARN the tools never returned is caught, including near-miss and truncated forms.
evals/readme.test.ts	17	Every npm run the README documents exists, and every count it quotes is true. Written after the README documented a script that did not exist.
evals/groundTruth.test.ts	17	Tier 1. Runs the fifteen checks in evals/checks.ts against a real scan of the mock, comparing with the hand-written answer key. No API key, no model, deterministic. Skipped without infrastructure.
aws/principal.test.ts	14	Session ARN → principal ARN for every identity shape, and that root and federated identities return a <i>reason</i> rather than a guess.
agent/cypherGuard.test.ts	13	Writes rejected, writes hidden in strings and comments rejected, legitimate reads allowed.
analysers/policy.test.ts	13	Effective <code>*</code> through managed, inline, wildcard and combination forms.
shared/scan.test.ts	13	<code>rollUpStatus()</code> — the verdict the whole partial-failure story rests on.
config.test.ts	10	Blank values treated as unset, and the configuration problems the UI reports.
agent/readOnlyGuard.test.ts	8	A change request always gets an explicit refusal; an answer that already declines is left alone.
graph/resourceArn.test.ts	8	That the router carries a full-length IAM ARN. Split deliberately: a config assertion that needs nothing and runs on every commit, plus an HTTP suite gated behind <code>SKIP_INTEGRATION</code> .
config.paths.test.ts	7	That no file addresses the filesystem with <code>URL.pathname</code> — a flat repository-wide ban, including in files that do not exist yet. The Windows <code>.env</code> bug.
remediation/fromGraph.test.ts	7	JSON-string properties parsed back, malformed ones dropped rather than thrown on.
analysers/reachability.test.ts	7	Multi-hop paths, and the public database that nothing can reach.
analysers/idle.test.ts	6	Structural waste, including the elastic IP whose unassociated state is an empty string.
aws/resourceExplorer.test.ts	6	That a missing index degrades to SDK enumeration instead of failing the scan.

aws/paginate.test.ts	4 That pagination actually paginates — including a second page nobody would notice was missing.
scan/limiter.test.ts	4 Concurrency is bounded, order is preserved, and a failing task does not stop the pool.
agent/toolLabels.test.ts	3 Every tool the model can call has user-facing copy, in both directions. The newest guard, and the smallest.

The two eval tiers

Tier 1 — is the data right? `evals/checks.ts` holds fifteen checks as data, so the same definitions run from Vitest and from the Trust panel in the product. No API key, no model, milliseconds.

Tier 2 — does the agent use it right? Twenty-one cases in `evals/cases.ts`, graded on cited ARNs with precision, recall and F1 rather than text similarity (`evals/grade.ts`). Needs a key; run with `npm run evals -w @sightline/api`. Current state: 21/21, mean F1 1.0.

WHY GRADE ON CITATIONS RATHER THAN WORDING

Citations are already validated against tool output, so they are a far stronger signal than string overlap — and they cannot be gamed by an answer that happens to use the right words. Two `mustNotMention` patterns were deleted during development for the opposite reason: a lexical pattern cannot test a negated claim, because "northwind-reports is *not* public" contains the string it was looking for.

12 If they ask X, open Y

The index to use live. Question on the left, the file to put on screen on the right.

If they ask...	Open
How does it get into a customer account?	<code>aws/credentials.ts</code> + <code>infra/readonly-role.yaml</code> (trust policy at the bottom)
Why not just use <code>ReadOnlyAccess</code> ?	<code>infra/readonly-role.yaml</code> — the <code>Deny</code> statement, and the header comments
What happens if a region is throttled?	<code>scan/runner.ts:71</code> (<code>explainFailure</code>) and <code>shared/scan.ts</code> (<code>rollUpStatus</code>)
How do you handle pagination?	Any collector — e.g. <code>collectors/ec2.ts</code> — plus <code>aws/paginate.test.ts</code>
Rate limits?	<code>aws/clients.ts</code> (<code>retryMode: "adaptive"</code>) and <code>scan/limiter.ts</code>
Why Neo4j and Postgres?	<code>db/schema.ts</code> next to <code>db/queries.ts:145</code> (<code>findNetworkPaths</code>)
Is this bucket public? How do you know?	<code>analysers/publicAccess.ts</code> , then the two identical fixture policies in <code>seed.ts</code>
What stops the agent changing something?	<code>agent/tools.ts</code> (no mutation expressible), <code>agent/cypherGuard.ts</code> , <code>agent/readOnlyGuard.ts</code> , the role's <code>Deny</code>
How do you know the agent isn't hallucinating?	<code>agent/citations.ts</code> — then the tool-call trail in the UI on a live answer
Why no agent framework?	<code>agent/agent.ts</code> — the whole loop is one readable file
How would you know if a change made it worse?	<code>evals/cases.ts</code> + <code>evals/grade.ts</code> , and the Trust panel in the running UI
What breaks on 10,000 resources?	<code>db/neo4j.ts:139</code> (whole-graph rebuild) and <code>graph/graph.controller.ts</code> (unpaginated graph fetch) — both named in the README
What changed since the last scan?	<code>db/repository.ts:271</code> (<code>diffScans</code> , fingerprint join) + <code>ChangesPanel.tsx</code>
How does the UI know a scan partly failed?	<code>ScanBanner.tsx</code> — the failed-units block, and <code>SCAN_FAULT_INJECTION</code> to demo it
Why is there no "fix it" button?	<code>remediation/remediation.ts</code> and <code>Remediation.tsx</code> — ADR-014
Is the mock data realistic?	<code>mock-aws/src/seed.ts</code> — built through the AWS SDK against moto, not hand-written JSON
How do you know the scanner is right?	<code>mock-aws/src/topology.ts</code> (hand-written answer key) + <code>evals/checks.ts</code>
What was the hardest bug?	<code>docs/ENGINEERING-LOG.md</code> #29 (admin analysis ignored IAM users) or #32 (the invented ARN)
Would this work on Windows?	<code>config.ts</code> (<code>fileURLToPath</code>) and <code>config.paths.test.ts</code>
What would you do next?	README, "What I would build next" — and the honest answer is the incremental scan

13 Worth knowing without looking it up

The numbers

Source files / lines (excl. tests)	80 files, ~13,200 lines
Tests	253 across 21 files
Agent tools	16, over 13 curated queries
Node kinds / edge types	20 / 13
Services collected	6 — EC2, VPC, S3, IAM, RDS, Lambda
Scan units in the demo	14 — four regional services × three regions, plus two global
Mock account	3 regions, ~101 resources, ~\$103/month of waste
Evals	Tier 1: 15 ground-truth checks. Tier 2: 21 cases, mean F1 1.0
ADRs / log entries	14 / 38

The five things that are true about the architecture

1. **Security reasoning is computed in code, never by the model.** Every verdict is a property on a node with a reason string attached, written during the scan.
2. **The agent cannot express a mutation.** Not "is told not to" — cannot. Four independent layers.
3. **Postgres is the system of record; the graph is disposable.** Rebuilt per scan, never written to directly.
4. **A scan does not fail.** It produces per-unit outcomes, and the UI is loud about the partial case.
5. **The mock differs from a real account in exactly one place.** An endpoint override in `aws/clients.ts`.

The honest limits, stated before anyone asks

- **Neo4j Community has no role-based access control**, so a read-only database *user* is not available — the read transaction is the guarantee. In production: an Enterprise read-only role or a replica.
- **The projection is a whole-graph rebuild.** Fine at a few thousand resources; the first thing to change for a large account is an incremental projection keyed on the fingerprints already in Postgres.
- **`GET /api/graph` is unpaginated.** The UI filters by kind, but the API hands over the whole graph — the next thing to break after the projection.
- **The IAM analyser is deliberately partial.** No permission boundaries, no SCPs, no condition-key evaluation. It says so in its own header, and it would rather miss an admin

than invent one.

- **Idle detection is structural, not usage-based.** Nothing here reads CloudWatch, so "unused" means "attached to nothing", which is a claim the data actually supports.

IF YOU ONLY REMEMBER ONE THING TO SAY ABOUT THE CODE

The interesting decisions are all the same decision: **put the judgement in code that can be tested, and leave the model to choose which tested thing to run.** That is the tool boundary, the computed verdicts, the validated citations, the generated-never-applied remediation, and the hand-written answer key. Everything else follows from it.